데이터 자동화 팩토리: ACCBRK 완벽 사용 가이드

복잡한 차량 동역학 시계열 데이터를 가장 직관적이고 완벽하게 분석하는 객체 지향(OOP) 파이프라인 매뉴얼.



데이터 흐름 & 입력 (Data Flow) 코어 클래스 & 제어 (Core Processing) 최종 리포트 & 출력 (Outputs)

복잡한 코드가 아닌, 하나의 거대한 공장으로 이해하기

Mini-Map

1 **공장장 (Manager)**
VT_ACCBRK
최상위 컨트롤 타워. 팩토리의 전체 기동 스위치를 켜고 끕니다. (전체 라이프사이클 통제)

2 **품질 관리 팀장 (QA Lead)**
VT_ACCBRK_PROJECT
여러 타이어의 성능을 비교하고, 외부 환경 변화를 보정합니다.



3 **핵심 조립 라인 (Assembly Line)**
VT_ACCBRK_TIRE
1회 주행 데이터를 독립된 캡슐로 포장하여 관리합니다.

4 **6대의 특수 로봇 팔 (Robotic Arms)**
AN_* 모듈들
가속도, 슬립률, 수막현상 등 특수 물리 연산을 전담합니다.

5 **출판소 (Renderer)**
PLOTTER_*
분석된 데이터를 대시보드 플롯 차트로 자동 생성합니다.

시스템 아키텍처 매트릭스: 제어와 흐름의 교차점



제어 계층 (Control Hierarchy)

VT_ACCBRK
(Top Level)

VT_ACCBRK_PROJECT
(Project Level)

VT_ACCBRK_TIRE
(Tire Level)

AN_* 서브 모듈
(Engine Level)

Step 1:
.vbo 원시 데이터
로드

Step 2:
유효 주행 구간(Valid
Segment) 추출

Step 3:
6대 물리 통계
지표 연산

Step 4:
타이어 집게 및
환경 선형 보정

Step 5:
백업(.tir, .pjt) 및
리포트 출력

데이터 흐름 (Data Lifecycle Timeline)

데이터 캡슐화: 단일 타이어 코어 클래스 (VT_ACCBRK_TIRE)







The Problem

원시 데이터(raw_df)는 노이즈가 많고 방대합니다.



The Solution

VT_ACCBRK_TIRE는 단 1회의 주행 데이터를 독립된 객체로 '포장(Encapsulation)'합니다.



Inside the Capsule

- df_raw: 필터링 전 원본 데이터.
- analysis(): 목표 속도(예: 80km/h) 내의 '급제동/가속 트리거 시점'을 자동 탐지하는 스캐너.
- df: 전처리가 완료된 깔끔한 '유효 구간' 데이터.
- clear_cache() & restore_cache(): 메모리를 관리하는 스마트 밸브.

Key Concept: 이 캡슐 하나만 있으면 분석, 백업(.tir 파일), 시각화가 모두 독자적으로 가능합니다.

환경 편차 상쇄: 프로젝트 관리자 (VT_ACCBRK_PROJECT)





Why do we need this?
아침과 저녁의 노면 온도/기상은 다릅니다.
동일한 타이어도 시간에 따라 성능이 변합니다
(Performance Drift).

선형 보정 (Linear Correction)

Step 1: 기준점 설정
ref_kwd = 'c' (콘트롤 타이어)를 지정하여 프로젝트에 영입(add_tire)합니다.

Step 2: 추적 (Tracking)
콘트롤 타이어의 시계열 추이를 선형 보간(Linear Interpolation)하여 환경 변화 '기준선(Baseline)'을 그립니다.

Step 3: 상쇄 (cal_corrected)
일반 타이어들의 성능을 이 기준선에 대비하여 '공정 성능 지수(Corrected Index)'로 환산합니다.



보정 전 (Before)



보정 후 (After)

6대 전문 물리 분석 모듈 (Sub-Physics Engines) 한눈에 보기



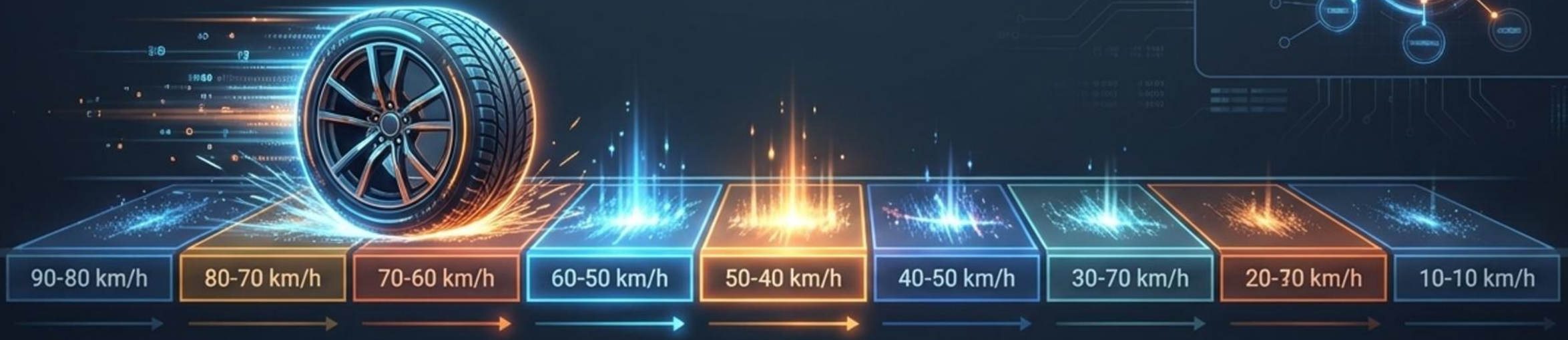
AN_ACCELERATION	[가속 트렌드] 속도 대역별 가속도 분산.	그립 유지 한계 진단.
AN_SPLIT_SPEED	[구간 분할] 임계 속도 기준 고속 vs 저속.	열 발생 및 제어 주파수 대조.
AN_SLIP_PDF	[ABS 효율] 슬립 확률 밀도 (KDE).	목표 점착 대역 점유율 확인.
AN_SLIP_MODE_SKEWNESS	[제어 편차] 2D 슬립-감속 왜도.	제어 안정성 기하학적 평가.
AN_MU_DROP	[수막 진단] K-Means & 결정 트리.	마찰력 저하 임계 속도 탐지.
AN_RESPONSE	[반응성] 제동 압력 대비 가속 반응 지연.	시스템 과도 응답성 평가.

이 서브 모듈들은 부모 타이어 객체에 바인딩되어 백그라운드에서 자동 기동됩니다.

Automated Data Factory

1. 가속도 트렌드 분석 (AN_ACCELERATION)





① 개념 (The Concept)

차량이 감속/가속할 때, 특정 속도 구간(interval)마다 타이어가 노면을 얼마나 꽉 쥐고 있는지(종방향 가속도 a_x)를 측정합니다.

② 주요 파라미터 & 출력

- interval = 10.0 (10km/h 간격으로 데이터를 쪼갭니다).
- 출력: 속도 구간별 평균, 표준편차를 담은 summary 데이터프레임.

③ 핵심 연산 (The Math)

평균 가속도: $\tilde{a}_k = \frac{1}{N_k} \sum_{i}^{n} a_i$

편차 트렌드 진단으로 속도 영역별 '그립 유지 한계'를 파악합니다.

Automated Data Factory

2. 고/저속 영역 분할 통계 (AN_SPLIT_SPEED)





개념 (The Concept)

고속과 저속은 타이어의 열 발생과 ABS 제어 주파수가 다릅니다. 하나의 기준으로 평가할 수 없기 때문에 시계열 데이터를 완벽히 양분하여 분석합니다.



파라미터 및 작동 방식



- 주요 파라미터: vel_split = 50.0 (분기 임계 속도 설정).
- 작동: 임계 속도 v_split을 기준으로 시계열 데이터를 High Region과 Low Region으로 분리합니다.
- 출력물: 각 영역에서의 총 제동 거리($S = \int v(t)dt$)와 평균 감속도를 대조하는 리포트.

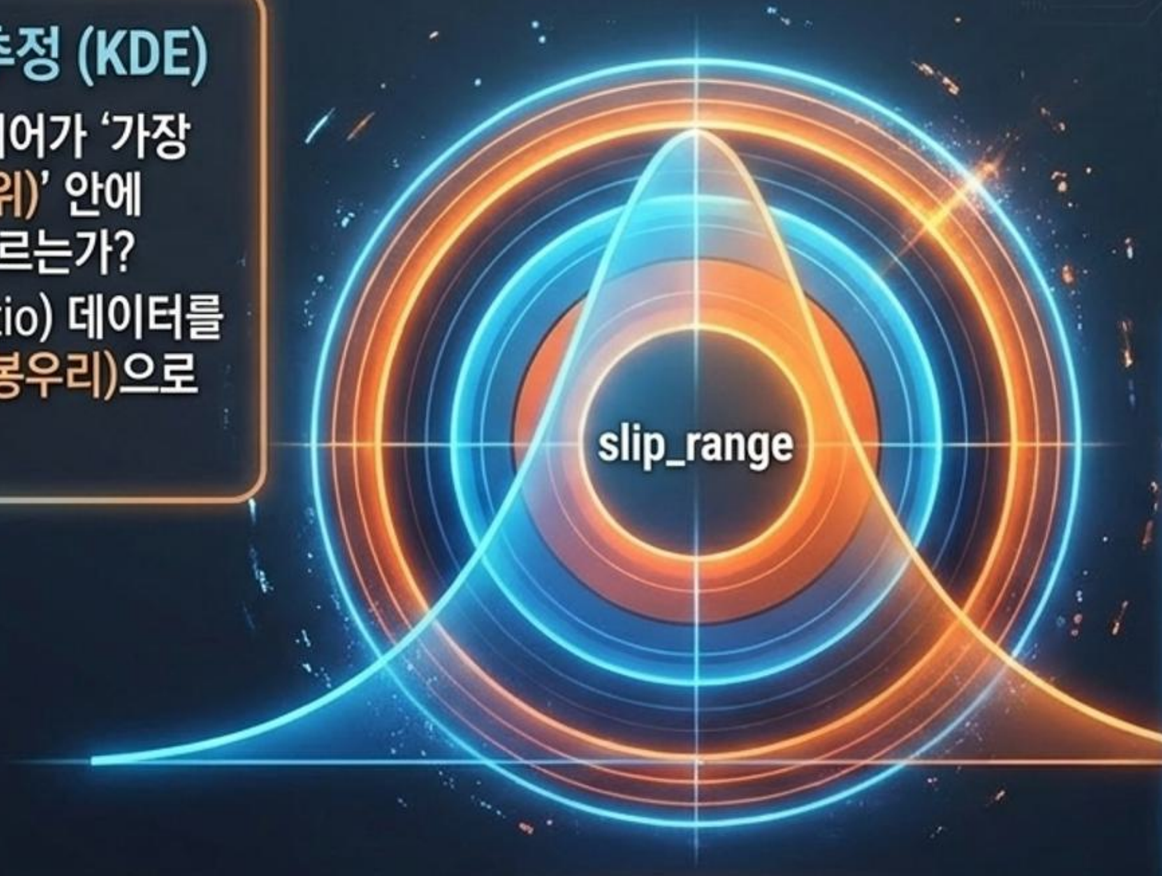
Automated Data Factory

3. ABS 제동 효율 추정 (AN_SLIP_PDF)



가우시안 커널 밀도 추정 (KDE)

- 개념: ABS 작동 시 타이어가 '가장 완벽한 접지력(타깃 범위)' 안에 얼마나 자주, 오래 머무르는가?
- 흩어진 슬립률(Slip Ratio) 데이터를 부드러운 확률 지형(산봉우리)으로 모델링합니다.

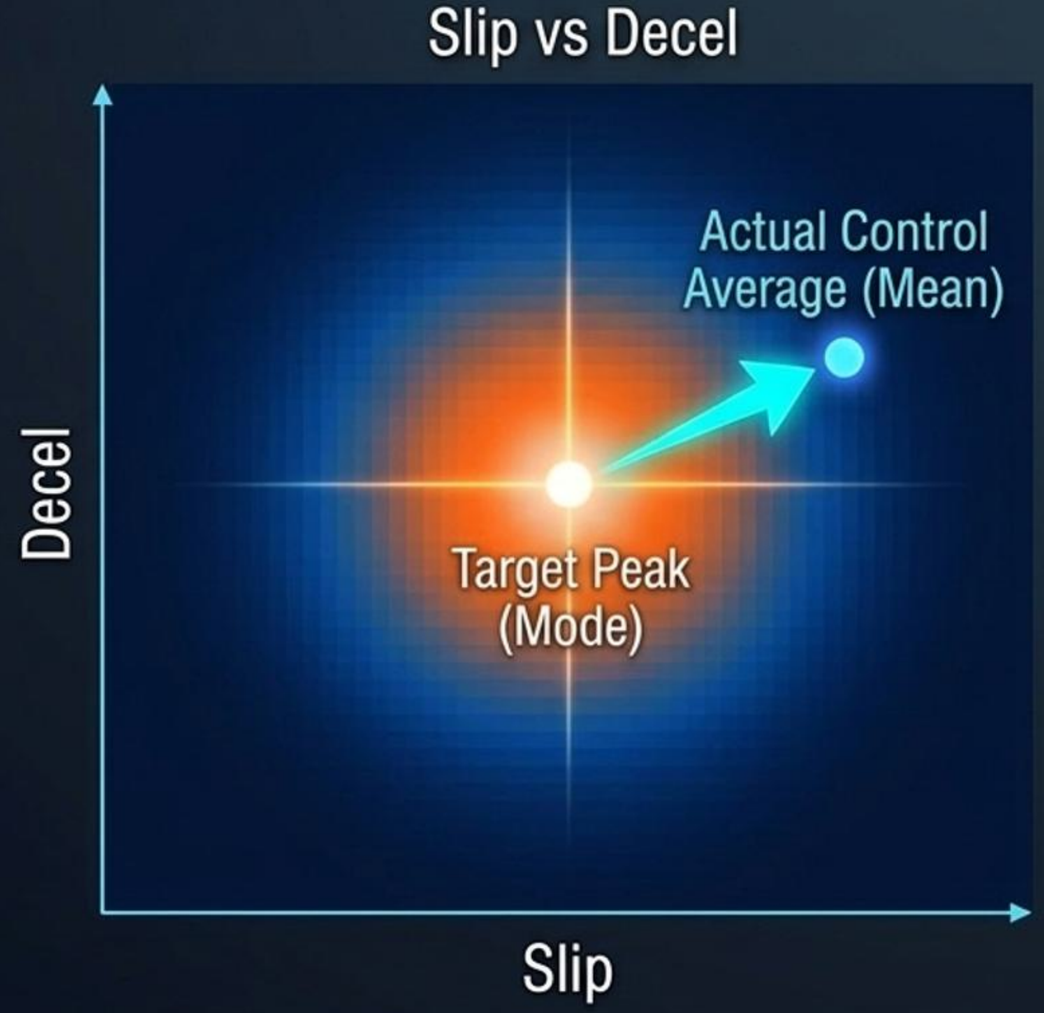


효율 점수 (Efficiency Score)

- 주요 파라미터: slip_range = [[0.0, 20.0]] (목표 점착 범위 – 우리의 과녁).
- 수식: Score = $\int f(s)ds \times 100$
- 전체 면적 중 우리의 과녁(타깃 범위) 안에 들어온 데이터의 확률 면적(%)을 점수로 계산합니다.

4. 2D 제어 편차 왜도 벡터 (AN_SLIP_MODE_SKEWNESS)





기하학적 오차 분석

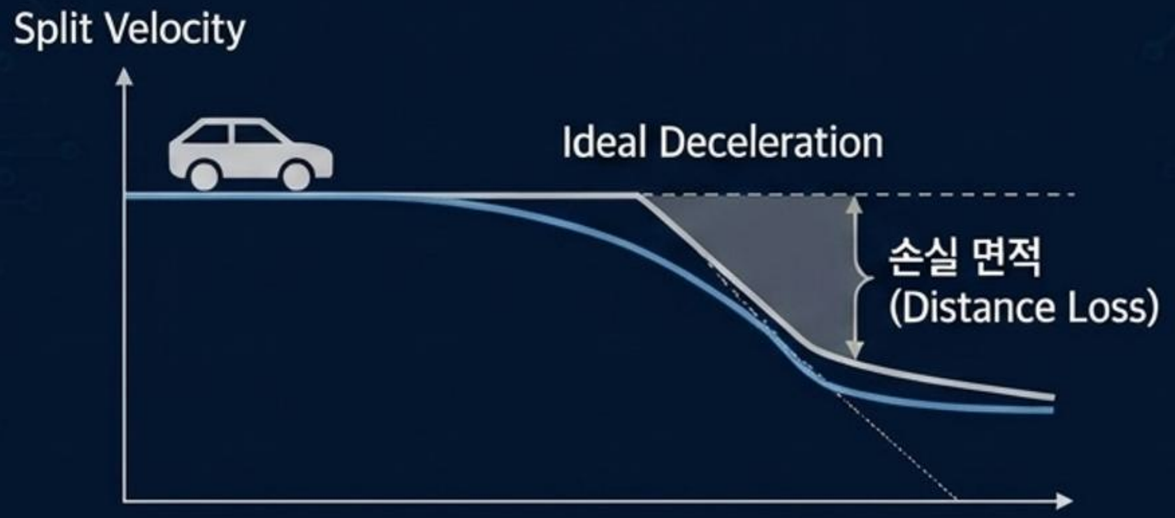
개념: 이상적인 '최고점(Mode)'과 실제 제어가 머무른 '평균점(Mean)'이 얼마나 빗나갔는가?

- 가로축 오차: $\Delta\,\text{Slip} = S_{mode} - S_{mean}$
- 세로축 오차: $\Delta\,\text{Decel} = A_{mode} - A_{mean}$
- 최단 유클리드 편차 거리: $D = \sqrt{\Delta\,\text{Slip}^2 + \Delta\,\text{Decel}^2}$

의미: 이 화살표(벡터)의 길이가 짧을수록, 방향이 흔들리지 않을수록 타이어의 제어 안정성이 뛰어남을 의미합니다.

Automated Data Factory

5. 수막현상 & 마찰 저하 임계 진단 (AN_MU_DROP)



개념 및 작동 방식 (AI 결합)

급제동 중 수막현상이나 마찰 포화로 타이어가 미끄러지기 시작하는 시점을 기계학습으로 찾아냅니다.

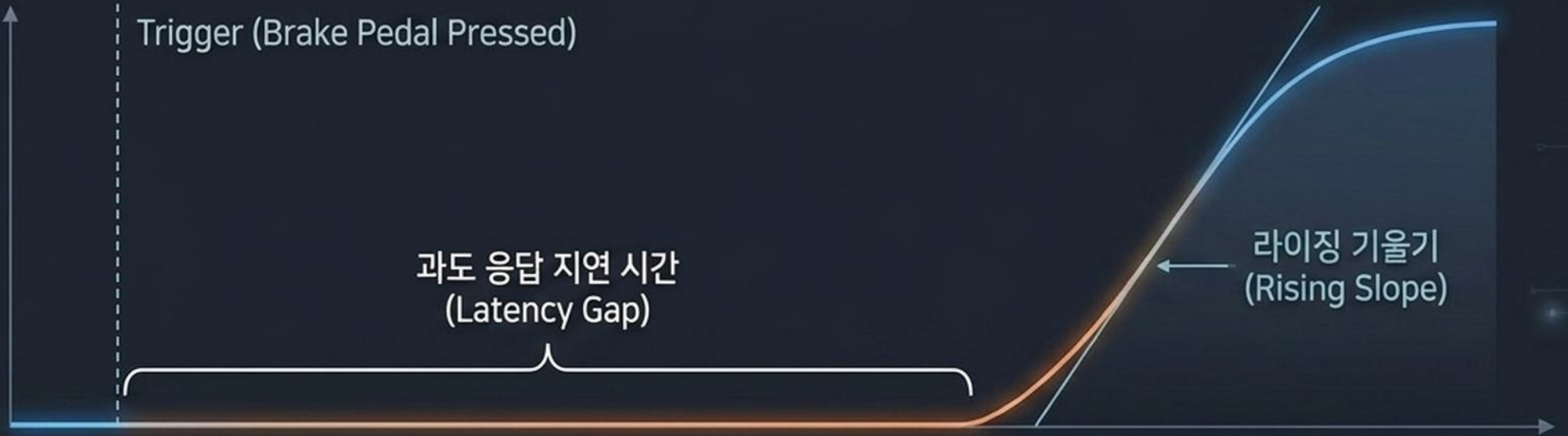
- 1차: K-Means 군집 분석으로 감속도 하락 구간 묶기.
- 2차: 의사결정나무(Decision Tree)로 가장 급격한 마찰 저하 노드 찾기.

제동거리 유실 정량화

이상적인 감속 궤적(선형)과 실제 미끄러진 궤적 사이의 면적을 적분하여 '손실된 제동거리(m)'를 정량 계산합니다.

Gemini Notebook

6. 제동 제어 지연 반응성 (AN_RESPONSE)



과도 응답 지연 시간 (Latency Gap)

운전자가 브레이크를 밟은 트리거 시점 대비 목표 감속도(response_target = −0.2g)에 도달하기까지 걸린 시간 차이(ms)를 계산합니다.

라이징 기울기 (Rising Slope)

지정된 감속 구간(rising_target)에서 제동력이 얼마나 빠르게 솟구치는지 선형 회귀 기울기(G/s)로 계산하여 시스템 초기 반응력을 진단합니다.

Automated Data Factory

시각화 렌더러 (PLOTTER_TIRE & PLOTTER_PROJECT)





Zero-Copy 렌더링

PLOTTER는 자체 데이터를 복사하지 않고, 부모 객체의 데이터를 약한 참조(weakref)로 투영하여 메모리 낭비 없이 차트를 그립니다.

단일 타이어 대시보드 (PLOTTER_TIRE)

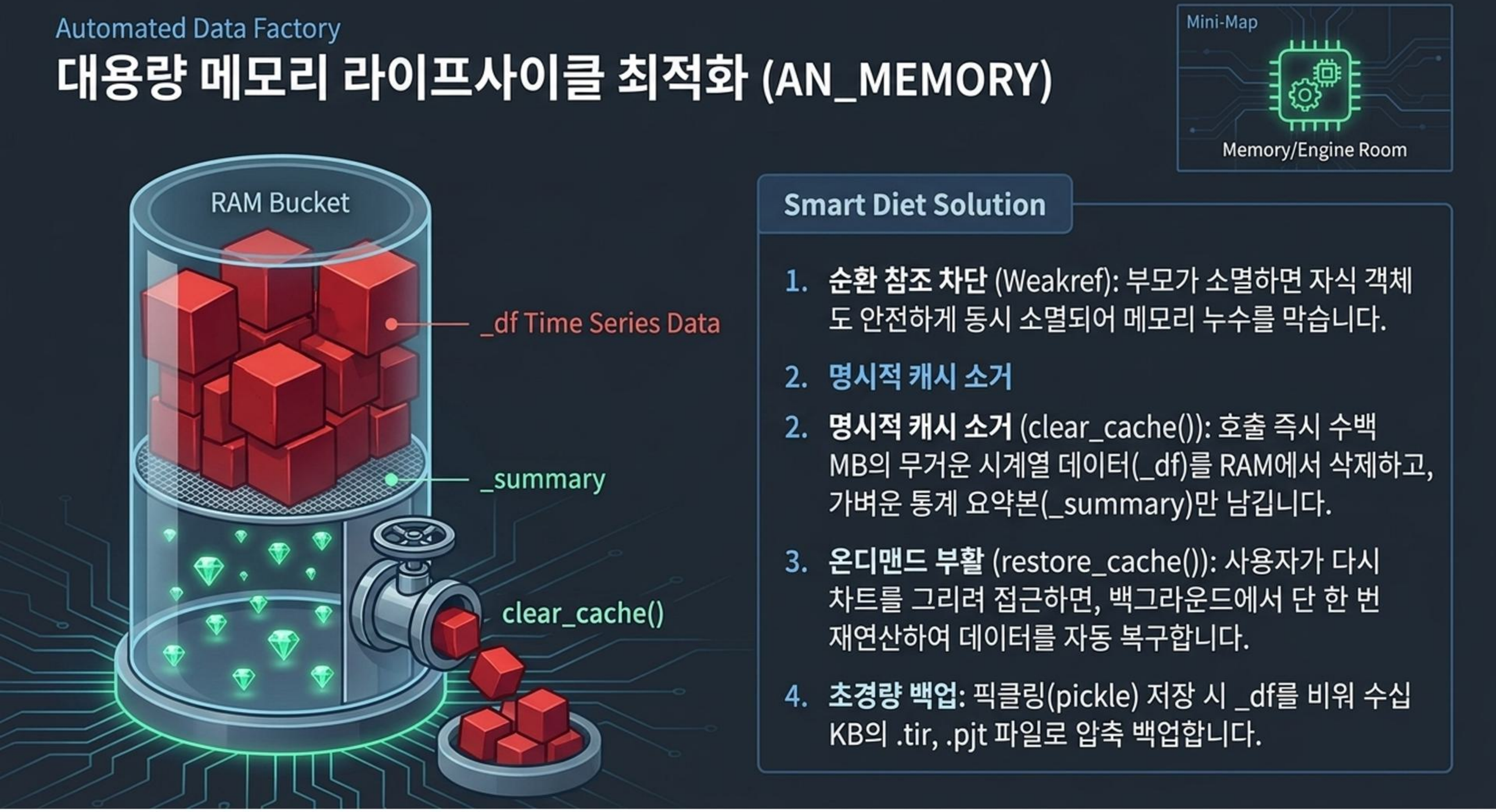
accbrk(), slip_pdf(), mu_drop() 등 단 한 줄의 코드로 8대 정밀 대시보드 .png / .html 자동 발행.

프로젝트 통합 대시보드 (PLOTTER_PROJECT)

plt(): 순수 절대값 기준 타이어 간 비교 차트.
plt_index(): 환경 편차가 100% 보정된 정밀 성능 지수(Corrected Index) 비교 바 차트.

Automated Data Factory

대용량 메모리 라이프사이클 최적화 (AN_MEMORY)



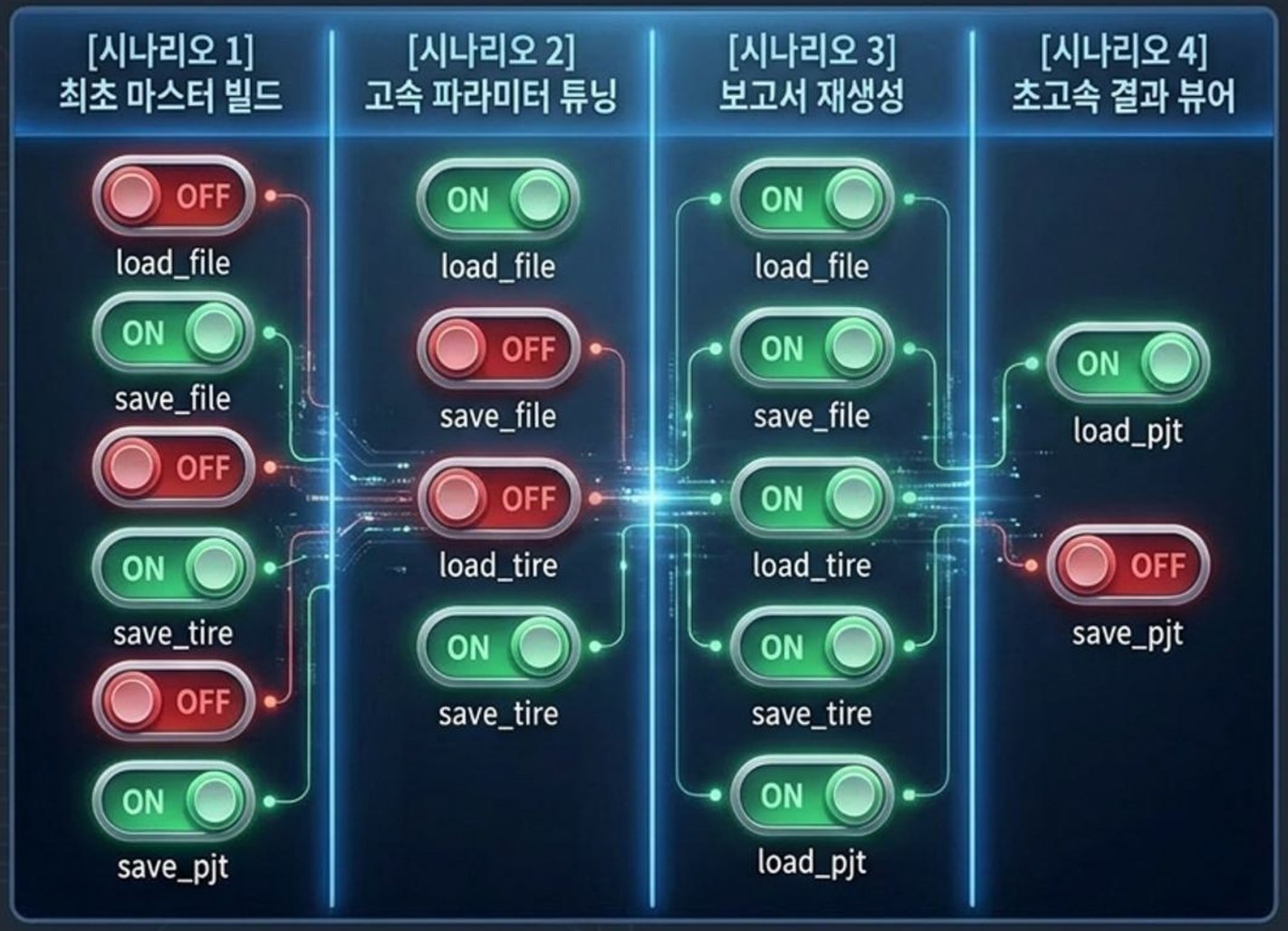
Smart Diet Solution

1. **순환 참조 차단** (Weakref): 부모가 소멸하면 자식 객체도 안전하게 동시 소멸되어 메모리 누수를 막습니다.
2. **명시적 캐시 소거**
2. **명시적 캐시 소거** (clear_cache()): 호출 즉시 수백 MB의 무거운 시계열 데이터(_df)를 RAM에서 삭제하고, 가벼운 통계 요약본(_summary)만 남깁니다.
3. **온디맨드 부활** (restore_cache()): 사용자가 다시 차트를 그리려 접근하면, 백그라운드에서 단 한 번 재연산하여 데이터를 자동 복구합니다.
4. **초경량 백업:** 픽클링(pickle) 저장 시 _df를 비워 수십 KB의 .tir, .pjt 파일로 압축 백업합니다.

Automated Data Factory

파이프라인 가동 시나리오 스위치보드





1. **마스터 빌드:** A to Z 원천 연산을 수행하고 전체 백업 생성.

2. **고속 튜닝:** 무거운 원시 파싱을 건너뛰고 .gz 캐시에서 즉시 로드하여 튜닝 조건만 재연산.

3. **보고서 재생성:** 물리 연산을 전면 우회하고 완성된 .pjt만 불러와 엑셀 즉시 재출간.

4. **초고속 뷰어:** 디스크 쓰기를 차단하고 메모리에 대시보드 화면만 띄움.

Automated Data Factory

실전 플레이북 1 & 2: 수동 제어 vs 원스톱 자동화

[Playbook 1] 단일 타이어 수동 제어 (매뉴얼 모드)

```
1  tire = VT_ACCBRK_TIRE(...)
2
3  tire.analysis(...)
4
5  tire.slip_pdf.analysis(...)
6
7  tire.clear_cache()
```

캡슐 생성

유효 구간 추출

6대 엔진 개별 튜닝 가동

메모리 비우기

[Playbook 2] 마법의 래퍼 (원스톱 자동화)

```
1
2  VT_ACCBRK_TIRE.run(df_raw, test_vel, ... save, clear)
```

해설: 단 한 줄의 코드로 전처리, 구간 검출, 6대 물리 연산, 플롯 저장, 캐시 오프로드까지 **모두 자동 처리**합니다.

Automated Data Factory

실전 플레이북 3: 중단 없는 대량 배치(Batch) 분석




```python
for data in list_datas:
    try:
        analyzer = VT_ACCBRK_TIRE.run(... test_vel, ...)
        analyzer.save(...)
        analyzer.clear_cache() # 필수 메모리 반환
    except Exception as e:
        logging.error("에러 핸들링")
        continue
```

문제 & 해결책

문제: 수백 개의 텔레메트리 파일 중 단 1개가 깨져 있으면 밤새 돌려놓은 스크립트가 멈춥니다.

해결책 (격벽 패턴 적용): 에러 발생 시 로그만 남기고 다음 파일로 넘어갑니다. 매 루프마다 clear_cache()를 호출하여 수백 개의 타이어를 분석해도 RAM 점유율은 일정하게 유지됩니다.

Automated Data Factory

실전 플레이북 4 & 5: 프로젝트 집계 및 이종 노면 병합

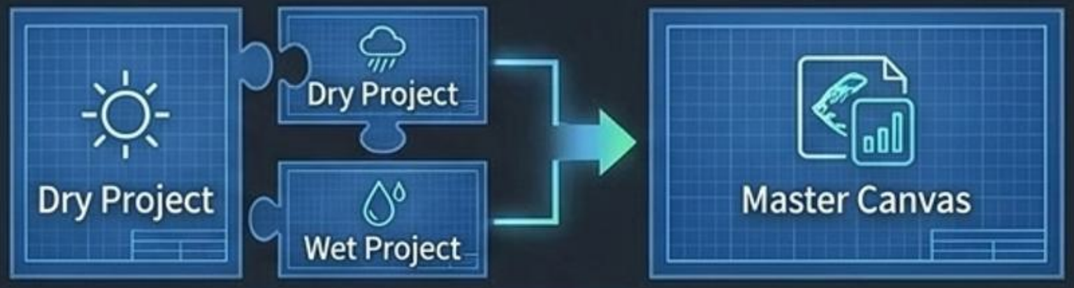
[Playbook 4] 선형 보정 구동

```
project = VT_ACCBRK_PROJECT(ref_kwd='Control')
project.add_tire(...)
project.cal_corrected()
```



[Playbook 5] 커스텀 조립 (VT_ACCBRK_merge)

```
VT_ACCBRK_merge(dict_pjt={'Dry': dry, 'Wet': wet})
```



해설: 과거 윈터 타이어 에셋(.tir)과 현재 프로토타입을 자유롭게 믹스 앤 매치하여 새로운 보고서를 즉시 조립합니다.

Automated Data Factory

실전 플레이북 6: 캐시 기반 초고속 파라미터 튜닝

Old Way



Full Reload of .csv / .vbo Estimated 25 min

Rapid Tuning Way



100%

Instant Load from .gz cache Sub-seconds

```
# 핵심 코드 설정
load_file = True   # 초경량 압축 캐시(.gz)를 1초 만에 로드
load_tire = False  # 타이어 연산은 변경된 튜닝 조건으로 재실행
save_pjt = True    # 새로운 마스터 리포트 덮어쓰기

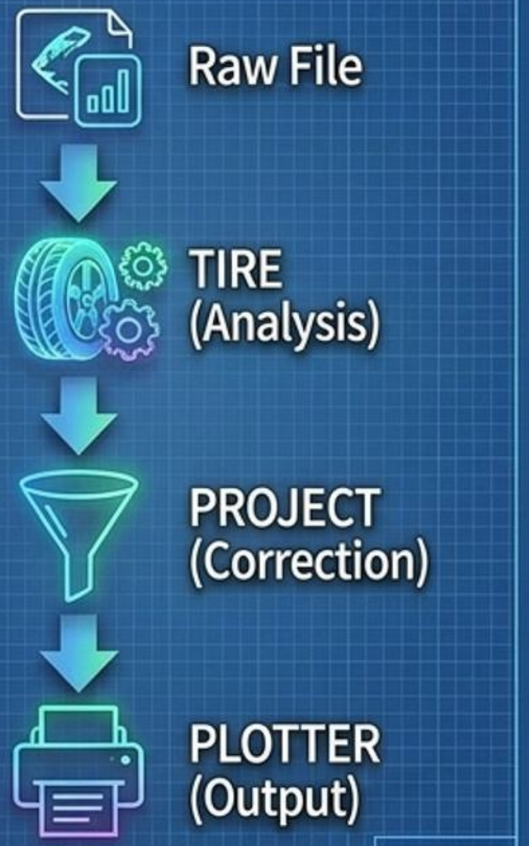
outlier = {'T1': [1, 5]}
```

개념: 이상치를 제거하거나 속도 구간을 바꿀 때마다
무거운 원본 데이터를 처음부터 다시 읽을 필요가 없습니다.
수 분이 걸리던 재분석 로딩 작업이 **단 몇 초 단위로 단축**됩니다.

Automated Data Factory

ACCBRK 마스터 맵: 전체 요약 레퍼런스

[기둥 1] 흐름 (Flow)



[기둥 2] 6대 코어 모듈 (Engines)

1. 가속도 (Accel)
2. 분할 (Split)
3. 효율 (PDF)
4. 편차 (Skewness)
5. 마찰저하 (Mu-Drop)
6. 응답성 (Response)

[기둥 3] 최적화 룰 (Golden Rules)

1. RAM이 부족할 땐 언제나 clear_cache()
2. 환경 편차 보정은 ref_kwd와 cal_corrected()
3. 재파싱 방지는 load_file=True

당신의 노트북은 이제 수백 기가의 텔레메트리 데이터를 다룰 수 있는 거대한 분석 팩토리입니다.